Week 2・Week 3 マニュアル

Week 1 で作ったブログに、以下の機能を追加します。

```
Week 2
  Step 1：コードブロックにコピーボタンをつける
  Step 2：.py ファイルのダウンロードボタンをつける
  Step 3：About ページを作る

Week 3
  Step 4：ナビゲーションバーを作る
  Step 5：タグページを作る
```

前提：Week 1 の状態

以下が完成している前提で進めます。

```
my-site/
├── app/
|   ├── articles/
|   |   ├── page.tsx
|   |   └── [slug]/page.tsx
|   ├── globals.css
|   ├── layout.tsx
|   └── page.tsx
├── content/articles/
|   └── python-sequential-files.mdx
├── lib/
|   └── articles.ts
└── public/
```

開発サーバーを起動しておきます。

```bash
cd ~/Projects/my-site
npm run dev
```

Week 2

Step 1：コードブロックにコピーボタンをつける

なぜ必要か

デフォルトのコードブロックはただテキストが表示されるだけです。
コピーボタンをつけることで、読者がコードをワンクリックで取得できるようになります。

1-1：components フォルダを作成

```bash
mkdir components
touch components/CodeBlock.tsx
```

1-2：CodeBlock.tsx を作成

`components/CodeBlock.tsx` に以下を貼り付けます。

```typescript
```

```
'use client'

import { useState } from 'react'

type Props = {
  children: string
  className?: string
  inline?: boolean
}

export default function CodeBlock({ children, className, inline }: Props) {
  const [copied, setCopied] = useState(false)

  if (inline) {
    return (
      <code className="bg-gray-100 text-gray-800 px-1.5 py-0.5 rounded text-sm font-
        {children}
      </code>
    )
  }

  const handleCopy = async () => {
    await navigator.clipboard.writeText(children)
    setCopied(true)
    setTimeout(() => setCopied(false), 2000)
  }

  return (
    <span className="relative block my-4">
      <pre className="bg-[#1e1e1e] text-[#d4d4d4] p-4 rounded-lg overflow-x-auto">
        <code className={className}>{children}</code>
      </pre>
      <button
        onClick={handleCopy}
        className="absolute top-3 right-3 text-xs px-2 py-1 rounded bg-gray-700 text
      >
        {copied ? 'コピー済み ✓' : 'コピー'}
      </button>
    </span>
  )
}
```

コードの説明

- `'use client'`：ボタンのクリックはブラウザ側の操作なので、クライアントコンポーネントとして宣言する

- `useState(false)`：コピー済みかどうかの状態を管理する
- `navigator.clipboard.writeText()`：クリップボードにテキストをコピーするブラウザのAPI
- `setTimeout(() => setCopied(false), 2000)`：2秒後にボタンを元に戻す
- `inline`：インラインコード（文中の`code`）とコードブロックを区別するフラグ

1-3：記事詳細ページに組み込む

`app/articles/[slug]/page.tsx`を以下に差し替えます。

```typescript
```

```
import { getAllArticles, getArticleBySlug } from '@/lib/articles'
import { MDXRemote } from 'next-mdx-remote/rsc'
import remarkGfm from 'remark-gfm'
import CodeBlock from '@/components/CodeBlock'

export async function generateStaticParams() {
  const articles = getAllArticles()
  return articles.map(article => ({ slug: article.slug }))
}

export default async function ArticlePage({
  params,
}: {
  params: Promise<{ slug: string }>
}) {
  const { slug } = await params
  const article = getArticleBySlug(slug)

  return (
    <main className="max-w-2xl mx-auto px-4 py-12">
      <div className="mb-8">
        <p className="text-sm text-gray-500 mb-2">
          {article.date} · {article.category}
        </p>
        <h1 className="text-3xl font-bold mb-4">{article.title}</h1>
        <div className="flex gap-2">
          {article.tags.map(tag => (
            <span
              key={tag}
              className="text-xs bg-gray-100 text-gray-600 px-2 py-1 rounded"
            >
              {tag}
            </span>
          ))}
        </div>
      </div>
      <article className="prose max-w-none">
        <MDXRemote
          source={article.content}
          options={{
            mdxOptions: {
              remarkPlugins: [remarkGfm],
            },
          }}
          components={{
            pre: ({ children }) => <>{children}</>,
```

```
            code: ({ className, children }) => {
              const isInline = !className
              return (
                <CodeBlock className={className} inline={isInline}>
                  {String(children).replace(/\n$/, '')}
                </CodeBlock>
              )
            },
          }}
        />
      </article>
    </main>
  )
}
```

> **components プロパティの役割** MDXRemote の `components` に渡すと、Markdown の要素をカスタムコンポーネントで上書きできます。`pre` を `<>{children}</>` にすることで pre タグを取り除き、`code` を CodeBlock に置き換えています。

ブラウザで記事詳細ページを開いてコードブロックの右上にコピーボタンが表示されれば成功です。

Step 2：.py ファイルのダウンロードボタンをつける

なぜ必要か

記事で紹介したPythonスクリプトを、読者が1クリックでダウンロードできるようにします。
`public/downloads/` に置いたファイルは `/downloads/ファイル名` というURLで公開されます。

2-1：ダウンロード用フォルダとファイルを作成

```bash
mkdir public/downloads
touch public/downloads/file_generator.py
```

`public/downloads/file_generator.py` に以下を貼り付けます。

```python
```

```python
import os

def generate_files(prefix, start, end, ext, output_dir="."):
    os.makedirs(output_dir, exist_ok=True)
    for i in range(start, end + 1):
        filename = f"{prefix}_{i:03d}.{ext}"
        filepath = os.path.join(output_dir, filename)
        with open(filepath, "w") as f:
            f.write(f"# {filename}\n")
    print(f"{end - start + 1}個のファイルを生成しました")

generate_files("data", 1, 10, "txt", "output")
```

2-2：DownloadButton コンポーネントを作成

```bash
touch components/DownloadButton.tsx
```

`components/DownloadButton.tsx` に以下を貼り付けます。

```typescript
export default function DownloadButton(props: { filename: string; label?: string })
  return (
    <a href={'/downloads/' + props.filename} className="inline-flex items-center gap
      {props.label ? props.label : props.filename}
    </a>
  )
}
```

> **注意点** JSXの中でテンプレートリテラル（バッククォート）を使うと環境によって文字化けすることがあります。そのため `href` は文字列結合（`+`）で書いています。

2-3：記事詳細ページに DownloadButton を追加

`app/articles/[slug]/page.tsx` の import と components を更新します。

```typescript
```

```
import { getAllArticles, getArticleBySlug } from '@/lib/articles'
import { MDXRemote } from 'next-mdx-remote/rsc'
import remarkGfm from 'remark-gfm'
import CodeBlock from '@/components/CodeBlock'
import DownloadButton from '@/components/DownloadButton'

export async function generateStaticParams() {
  const articles = getAllArticles()
  return articles.map(article => ({ slug: article.slug }))
}

export default async function ArticlePage({
  params,
}: {
  params: Promise<{ slug: string }>
}) {
  const { slug } = await params
  const article = getArticleBySlug(slug)

  return (
    <main className="max-w-2xl mx-auto px-4 py-12">
      <div className="mb-8">
        <p className="text-sm text-gray-500 mb-2">
          {article.date} · {article.category}
        </p>
        <h1 className="text-3xl font-bold mb-4">{article.title}</h1>
        <div className="flex gap-2">
          {article.tags.map(tag => (
            <span
              key={tag}
              className="text-xs bg-gray-100 text-gray-600 px-2 py-1 rounded"
            >
              {tag}
            </span>
          ))}
        </div>
      </div>
      <article className="prose max-w-none">
        <MDXRemote
          source={article.content}
          options={{
            mdxOptions: {
              remarkPlugins: [remarkGfm],
            },
          }}
          components={{
```

```
            pre: ({ children }) => <>{children}</>,
            code: ({ className, children }) => {
              const isInline = !className
              return (
                <CodeBlock className={className} inline={isInline}>
                  {String(children).replace(/\n$/, '')}
                </CodeBlock>
              )
            },
            DownloadButton,
          }}
        />
      </article>
    </main>
  )
}
```

2-4：MDX 記事にダウンロードボタンを追加

`content/articles/python-sequential-files.mdx` の `## はじめに` の直後に以下を追加します。

```mdx
<DownloadButton filename="file_generator.py" label="file_generator.py をダウンロード"
```

ブラウザで記事を開いてダウンロードボタンが表示されれば成功です。

> **新しい記事にダウンロードボタンをつける手順**
>
> 1. `public/downloads/` に .py ファイルを置く
> 2. MDX 記事の中に `<DownloadButton filename="ファイル名.py" label="説明文" />` を書く これだけで完了です。

Step 3：About ページを作る

```bash
mkdir app/about
touch app/about/page.tsx
```

ターミナルで以下を実行してファイルを作成します。

```bash
```

```
python3 << 'PYEOF'
content = '''export default function AboutPage() {
  const techs = ['Python', 'Next.js', 'TypeScript', 'Tailwind CSS', 'Linux', 'Git']
  return (
    <main className="max-w-2xl mx-auto px-4 py-12">
      <h1 className="text-3xl font-bold mb-8">About</h1>
      <section className="mb-10">
        <h2 className="text-xl font-semibold mb-3">このサイトについて</h2>
        <p className="text-gray-600 leading-relaxed">
          研究・開発の過程で得た知識を整理・再利用するためのナレッジベースです。
          Python自動化・環境構築・情報系理論・研究効率化に関する記事とツールを公開しています。
          掲載しているコードはすべてコピー・ダウンロードして使えます。
        </p>
      </section>
      <section className="mb-10">
        <h2 className="text-xl font-semibold mb-3">自己紹介</h2>
        <p className="text-gray-600 leading-relaxed">
          大学院で研究をしながらWeb開発・Python自動化に取り組んでいます。
          CFDやデータ解析を効率化するツールを作りながら、学んだことをこのサイトにまとめています。
        </p>
      </section>
      <section className="mb-10">
        <h2 className="text-xl font-semibold mb-3">技術スタック</h2>
        <div className="flex flex-wrap gap-2">
          {techs.map(tech => (
            <span key={tech} className="bg-gray-100 text-gray-700 px-3 py-1 rounded-
              {tech}
            </span>
          ))}
        </div>
      </section>
      <section>
        <h2 className="text-xl font-semibold mb-3">リンク</h2>
        <div className="flex gap-4">
          <a href="https://github.com/Baba-1811" target="_blank" rel="noopener noref
        </div>
      </section>
    </main>
  )
}
'''
with open('app/about/page.tsx', 'w') as f:
    f.write(content)
print('done')
PYEOF
```

> **なぜ python3 で書き出すのか** JSXの中に `<a href="...">` のような `<` や `>` を含むコードをターミナルに貼り付けると、シェルが特殊文字として解釈して文字が消えることがあります。Python で文字列として書き出すことでこの問題を回避できます。

`http://localhost:3000/about` でAboutページが表示されれば成功です。

Week 3

Step 4：ナビゲーションバーを作る

なぜ必要か

現状はURLを直接打たないと各ページに移動できません。ナビバーを `app/layout.tsx` に追加することで全ページ共通のヘッダーが表示されます。

`app/layout.tsx` を以下に差し替えます。

```typescript
```

```
import type { Metadata } from 'next'
import { Geist, Geist_Mono } from 'next/font/google'
import Link from 'next/link'
import './globals.css'

const geistSans = Geist({
  variable: '--font-geist-sans',
  subsets: ['latin'],
})

const geistMono = Geist_Mono({
  variable: '--font-geist-mono',
  subsets: ['latin'],
})

export const metadata: Metadata = {
  title: '研究・開発効率化ナレッジベース',
  description: 'Python自動化・環境構築・情報系理論・研究効率化に関する技術記事とツール',
}

export default function RootLayout({
  children,
}: {
  children: React.ReactNode
}) {
  return (
    <html lang="ja">
      <body className={`${geistSans.variable} ${geistMono.variable} antialiased`}>
        <header className="border-b border-gray-200">
          <nav className="max-w-2xl mx-auto px-4 py-4 flex items-center justify-betw
            <Link href="/" className="font-bold text-lg">
              KB
            </Link>
            <div className="flex gap-6 text-sm">
              <Link href="/articles" className="text-gray-600 hover:text-black trans
                記事
              </Link>
              <Link href="/tools" className="text-gray-600 hover:text-black transiti
                ツール
              </Link>
              <Link href="/about" className="text-gray-600 hover:text-black transiti
                About
              </Link>
            </div>
          </nav>
        </header>
```

```
        {children}
      </body>
    </html>
  )
}
```

> **layout.tsx の役割** Next.js の App Router では `layout.tsx` が全ページの「枠」になります。`{children}` の部分に各ページのコンテンツが入ります。ここにナビバーを書くことで、全ページに自動で表示されます。

全ページの上部にナビバーが表示されれば成功です。

Step 5：タグページを作る

なぜ必要か

記事一覧のタグをクリックしたとき、そのタグがついた記事だけを表示するページです。
記事が増えたときに探しやすくなります。

5-1：タグページを作成

```bash
mkdir -p "app/tags/[tag]"
touch "app/tags/[tag]/page.tsx"
```

`app/tags/[tag]/page.tsx` に以下を貼り付けます。

```typescript
```

```
import Link from 'next/link'
import { getAllArticles, getAllTags } from '@/lib/articles'

export async function generateStaticParams() {
  const tags = getAllTags()
  return tags.map(tag => ({ tag }))
}

export default async function TagPage({
  params,
}: {
  params: Promise<{ tag: string }>
}) {
  const { tag } = await params
  const articles = getAllArticles().filter(a => a.tags.includes(tag))

  return (
    <main className="max-w-2xl mx-auto px-4 py-12">
      <div className="mb-8">
        <Link href="/articles" className="text-sm text-gray-500 hover:underline">
          記事一覧に戻る
        </Link>
        <h1 className="text-3xl font-bold mt-2">#{tag}</h1>
        <p className="text-gray-500 text-sm mt-1">{articles.length}件</p>
      </div>
      <ul className="space-y-4">
        {articles.map(article => (
          <li key={article.slug} className="border border-gray-200 rounded-lg p-5">
            <Link href={`/articles/${article.slug}`}>
              <h2 className="font-semibold hover:underline">{article.title}</h2>
            </Link>
            <p className="text-sm text-gray-500 mt-1">{article.date} · {article.cate
            <p className="text-gray-600 text-sm mt-2">{article.excerpt}</p>
          </li>
        ))}
      </ul>
    </main>
  )
}
```

5-2：記事一覧のタグをリンクにする

`app/articles/page.tsx` を以下に差し替えます。

```typescript
```

```
import Link from 'next/link'
import { getAllArticles } from '@/lib/articles'

export default function ArticlesPage() {
  const articles = getAllArticles()

  return (
    <main className="max-w-2xl mx-auto px-4 py-12">
      <h1 className="text-3xl font-bold mb-8">記事一覧</h1>
      <ul className="space-y-6">
        {articles.map(article => (
          <li key={article.slug} className="border border-gray-200 rounded-lg p-6">
            <Link href={`/articles/${article.slug}`}>
              <h2 className="text-xl font-semibold hover:underline">
                {article.title}
              </h2>
            </Link>
            <p className="text-sm text-gray-500 mt-1">{article.date} · {article.cate
            <p className="text-gray-700 mt-2">{article.excerpt}</p>
            <div className="flex gap-2 mt-3">
              {article.tags.map(tag => (
                <Link
                  key={tag}
                  href={`/tags/${tag}`}
                  className="text-xs bg-gray-100 text-gray-600 px-2 py-1 rounded hov
                >
                  #{tag}
                </Link>
              ))}
            </div>
          </li>
        ))}
      </ul>
    </main>
  )
}
```

記事一覧のタグが`#automation`のようにリンクになり、クリックするとタグページに飛べれば成功です。

最後：GitHubにpushしてVercelに反映

```bash
```

```
git add .
git commit -m "add: copy button, download button, about page, navbar, tag pages"
git push
```

pushするとVercelが自動でビルド・デプロイします。
1〜2分後に公開URLで確認してください。

完成したフォルダ構成

```
my-site/
├── app/
|   ├── about/
|   |   └── page.tsx            ← About ページ（Week 2）
|   ├── articles/
|   |   ├── page.tsx             ← 記事一覧（タグリンク追加）
|   |   └── [slug]/page.tsx     ← 記事詳細（コピー・DLボタン追加）
|   ├── tags/
|   |   └── [tag]/page.tsx      ← タグページ（Week 3）
|   ├── globals.css
|   ├── layout.tsx              ← ナビバー追加（Week 3）
|   └── page.tsx
├── components/
|   ├── CodeBlock.tsx           ← コピーボタン（Week 2）
|   └── DownloadButton.tsx      ← ダウンロードボタン（Week 2）
├── content/articles/
|   └── python-sequential-files.mdx
├── lib/
|   └── articles.ts
└── public/
    └── downloads/
        └── file_generator.py   ← ダウンロード用ファイル（Week 2）
```

トラブルシューティング

コピーボタンが表示されない `CodeBlock.tsx` の先頭に `'use client'` があるか確認してください。ないとクリックイベントが動きません。

ダウンロードボタンをクリックしてもダウンロードされない `public/downloads/` にファイルが存在するか確認してください。`ls public/downloads/` で確認できます。

タグページが404になる `app/tags/[tag]/page.tsx` に `generateStaticParams` が定義されているか確認してください。これがないとNext.jsがタグページを生成しません。

JSXで`<`や`>`を含むコードを貼り付けるとエラーになる ターミナルへの貼り付けで`<`が消えることがあります。 その場合はPythonでファイルを書き出す方法を使ってください。

```bash
python3 << 'PYEOF'
content = '''ここにコードを書く'''
with open('ファイルパス', 'w') as f:
    f.write(content)
PYEOF
```

`npm run dev`でポートが3001になる 別のターミナルで古い`npm run dev`が動いています。 プロセスIDを確認して終了させてください。

```bash
lsof -i :3000        # ポート3000を使っているプロセスを確認
kill プロセスID      # 終了させる
npm run dev          # 再起動
```